Architecture des Ordinateurs : Représentation des Entiers et Arithmétique en Complément à Deux

Thème principal : Représentation des entiers signés, algorithme du complément à deux, arithmétique binaire bas niveau et détection d'Overflow

Synthèse exécutive : Ce cours aborde les mécanismes fondamentaux utilisés par les ordinateurs pour coder et manipuler les entiers signés en mémoire. Face à l'impossibilité matérielle d'enregistrer un symbole '+' ou '-' sur des circuits électroniques bistables (qui ne stockent que 0V ou Vcc), l'architecture informatique moderne repose sur la convention du complément à deux sur N bits. Ce système résout l'inconvénient du double zéro (+0 et -0) présent dans la représentation 'Signe et Magnitude' et permet d'unifier l'addition et la soustraction au sein du même circuit additionneur dans l'Unité Arithmétique et Logique (ALU). Le cours étudie la méthode de conversion en 3 étapes, les plages de valeurs dissymétriques, l'interprétation du bit de poids fort (MSB), la gestion de la retenue sortante (Carry Out) et la règle formelle de détection du dépassement de capacité (Overflow).

1. Problématique matérielle et justification de la convention du Complément à Deux

Au niveau le plus bas de l'architecture matérielle d'un processeur, la mémoire vive et les registres sont composés de circuits électroniques bistables (bascules). Ces composants ne reconnaissent que deux états de tension électrique : 0 Volt (niveau logique 0) et la tension d'alimentation Vcc (niveau logique 1). Un microprocesseur ne possède aucun moyen physique d'enregistrer directement un glyphe abstrait comme '+' ou '-'. Pour représenter des entiers relatifs, les architectes ont d'abord expérimenté le codage 'Signe et Magnitude', où le bit le plus à gauche (MSB) indique le signe (0 pour positif, 1 pour négatif) et les bits suivants portent la valeur absolue. Cette approche présente deux faiblesses majeures : elle crée deux représentations distinctes pour la valeur zéro (+0 et -0), ce qui gaspille une combinaison binaire utile et alourdit la logique matérielle lors des tests d'égalité ; d'autre part, elle oblige à construire deux circuits séparés dans l'Unité Arithmétique et Logique (ALU), l'un pour additionner et l'autre pour soustraire. La convention du complément à deux résout intégralement ces contraintes : le nombre zéro y possède une signature binaire unique (tous les bits à 0) et l'opération de soustraction A - B s'effectue sous la forme d'une addition A + (-B), réutilisant directement l'additionneur binaire standard de l'ALU sans composant additionnel.

- Absence de signe physique : Les bascules et transistors de la mémoire ne stockent que deux états logiques (0 pour 0 Volt, 1 pour Vcc). Il n'existe aucun composant pour le symbole '+' ou '-'.
- Échec du codage Signe et Magnitude : Cette méthode génère deux représentations pour zéro (+0 = 0000_2 et -0 = 1000_2 sur 4 bits), ce qui gaspille une combinaison binaire et impose deux circuits distincts dans l'ALU.
- Unification par le Complément à Deux : Cette convention attribue au zéro une représentation unique (0000...0) et transforme la soustraction A - B en une addition A + (-B), exécutée par le même additionneur matériel dans l'ALU.

Cas concret enseignant : *La métaphore du compteur kilométrique d'une voiture ancienne à 4 chiffres : si le compteur affiche 0000 km et que vous effectuez un kilomètre en marche arrière, le compteur repasse mécaniquement à 9999. En binaire sur 8 bits, faire 00000000_2 - 1 donne 11111111_2, ce qui constitue la définition formelle de -1 en complément à deux.*

2. Algorithme de Conversion et Plage de Valeurs sur N bits

Pour convertir un entier négatif -X en complément à deux, il est impératif de définir au préalable la largeur N du mot mémoire (ex: 8, 16, 32 ou 64 bits). La méthode exige d'exécuter rigoureusement trois étapes dans l'ordre : 1) Convertir la valeur absolue X en binaire naturel et la cadrer sur N bits en complétant par des 0 sur la gauche ; 2) Inverser la totalité des bits (les 0 deviennent 1 et les 1 deviennent 0), ce qui produit le complément à un ; 3) Ajouter 1 au mot binaire obtenu pour former le complément à deux. Le bit le plus à gauche (MSB) vaut 0 pour un nombre positif ou nul, et 1 pour un nombre négatif. En raison du rattachement du zéro (0000...0, dont le MSB vaut 0) au domaine des nombres positifs, la plage de valeurs représentables sur N bits est dissymétrique : la plage comporte

toujours une valeur négative de plus que de valeurs positives, s'étendant formellement de -2^(N-1) à +(2^(N-1) - 1). Sur 8 bits, la plage s'étend ainsi de -128 à +127.

- Algorithme des 3 étapes : Pour coder -X, 1) Écrire X en binaire pur sur N bits complets ; 2) Inverser chaque bit (Complément à 1) ; 3) Ajouter 1 au résultat (Complément à 2).
- Rôle du MSB (Most Significant Bit) : Le bit de poids fort (le plus à gauche) sert d'indicateur de signe (0 = positif ou nul, 1 = négatif).
- Plage de valeurs dissymétrique : Sur N bits, les valeurs représentables s'étendent de -2^(N-1) à +(2^(N-1) - 1). Sur 8 bits, cela correspond à [-128, +127].

Cas concret enseignant : *Avertissement crucial du professeur pour les contrôles et partiels : il est IMPÉRATIF de fixer la taille N bits (ex: 8 bits) et d'écrire les zéros de cadrage à gauche AVANT d'effectuer l'inversion des bits. Inverser la séquence '1100' (+12 sur 4 bits) sans ajouter au préalable les 4 zéros de cadrage '00001100' donne '0011' au lieu de '11110011', ce qui conduit à un résultat totalement faux en examen !*

3. Arithmétique Binaire, Retenue (Carry Out) et Détection d'Overflow

Lors de l'exécution d'une addition binaire dans un registre processeur de taille fixe N, deux indicateurs logiques distincts doivent être analysés au niveau du registre d'état (drapeaux/flags) : le Carry Out (C) et l'Overflow (OV). Le Carry Out représente la retenue sortante générée au-delà du bit MSB (à la position N+1). En arithmétique signée en complément à deux, cette retenue sortante est rejetée car le processeur effectue ses opérations en arithmétique modulaire de période 2^N. L'Overflow, en revanche, signale une erreur de dépassement de capacité où le bit de signe a été corrompu par le calcul arithmétique. La règle d'évaluation formelle pour détecter la présence d'un Overflow est stricte : un Overflow se produit si et seulement si deux opérandes ayant le même bit de signe (deux positifs ou deux négatifs) sont additionnés et donnent un résultat avec un bit de signe opposé. Si les deux opérandes sont de signes différents (un positif et un négatif), la survenue d'un Overflow est mathématiquement et physiquement impossible.

- Soustraction unifiée : Calculée directement sous la forme A - B = A + (-B) par l'ALU.
- Retenue sortante (Carry Out) : Retenue générée au-delà du bit MSB, systématiquement ignorée en arithmétique signée en complément à deux car le processeur travaille modulo 2^N.
- Drapeau d'Overflow (OV) : Indique un dépassement de capacité où le résultat sort de la plage autorisée [-2^(N-1), +2^(N-1)-1].
- Règle d'or de l'Overflow : Un Overflow survient si et seulement si l'addition de deux opérandes de même signe (deux positifs ou deux négatifs) produit un résultat de signe opposé.

Cas concret enseignant : *L'anecdote historique du crash d'Ariane 5 (vol 501 en 1996) : une conversion de type d'un nombre flottant sur 64 bits vers un entier signé sur 16 bits sans protection contre le dépassement de capacité a provoqué un Overflow non capturé. L'Unité de Référence Inertielle a transmis une valeur corrompue au calculateur de bord, interprétée à tort comme une déviation de trajectoire, provoquant le braquage brutal des tuyères puis l'autodestruction de la fusée.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la retenue tout à gauche (Carry sortant du bit MSB) est-elle systématiquement jetée lors de l'addition en complément à deux ?
Réponse du professeur : Parce que les registres matériels ont une taille fixe de N bits et travaillent en arithmétique modulaire modulo 2^N. Dans le système du complément à deux, le 'dépassement' au-delà du N-ième bit fait naturellement boucler la somme sur le bon résultat signé. La retenue sortante (Carry) est pertinente en arithmétique non signée, mais doit être ignorée lors du calcul du résultat en complément à deux.

Question étudiant : Comment l'ordinateur fait-il la différence en mémoire entre le mot binaire 11111111 qui vaut 255 en non-signé et -1 en complément à deux ?
Réponse du professeur : L'ordinateur ne fait aucune différence au niveau du stockage brut : dans la mémoire ram, 11111111 n'est qu'une suite de 8 niveaux logiques d'électricité. C'est le contexte d'exécution (le type de variable déclaré en C++ comme uint8_t vs int8_t, ou l'instruction processeur utilisée) qui indique au programme si cette suite de bits doit être interprétée comme un entier non signé (255) ou comme un entier signé en complément à deux (-1).

■ Exercice & Démonstration d'Application

Énoncé : Sur une architecture 8 bits, effectuer la soustraction signée : 35 - 75. Détailler le codage binaire des deux opérandes en complément à deux, poser l'addition binaire correspondante, donner le résultat décimal final et analyser la valeur du drapeau d'Overflow (OV).

Démarche de résolution :
Étape 1 : Codage du premier opérande (+35) sur 8 bits. 35 en binaire pur = 32 + 2 + 1 = 00100011_2. MSB = 0 (nombre positif). Étape 2 : Codage du second opérande (-75) en complément à deux sur 8 bits. - Valeur absolue (+75) sur 8 bits : 64 + 8 + 2 + 1 = 01001011_2. - Inversion des bits (Complément à 1) : 10110100_2. - Ajout de 1 (Complément à 2) : 10110100 + 1 = 10110101_2 (Représentation de -75 sur 8 bits). Étape 3 : Pose et exécution de l'addition binaire (+35) + (-75). 00100011 (+35) + 10110101 (-75) ---------- 11011000_2 Étape 4 : Interprétation du résultat binaire 11011000_2. Le bit de poids fort MSB vaut 1, le résultat est négatif. Pour retrouver sa valeur absolue décimale, on réapplique le complément à deux sur 11011000_2 : - Inversion des bits : 00100111_2 - Ajout de 1 : 00100111 + 1 = 00101000_2 = 32 + 8 = 40 décimal. Le mot binaire 11011000_2 représente donc la valeur -40 en décimal. Étape 5 : Analyse du drapeau d'Overflow (OV). On additionne un nombre positif (+35, MSB=0) et un nombre négatif (-75, MSB=1). Les deux opérandes possédant des bits de signe différents, l'apparition d'un Overflow est mathématiquement impossible.

Résultat : Résultat binaire final : 11011000_2 (égal à -40 en décimal). Drapeau d'Overflow OV = 0 (aucun dépassement de capacité). Le calcul 35 - 75 = -40 est exact et validé.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la plage exacte des entiers signés représentables sur un mot mémoire de 16 bits en complément à deux ?

R : La plage s'étend de -2^(16-1) à +(2^(16-1) - 1), soit de -32 768 à +32 767.

Q : Comment détecte-t-on de façon certaine la présence d'un Overflow lors de l'addition de deux entiers signés en examen ?

R : Un Overflow survient si et seulement si l'addition de deux opérandes ayant le même signe (deux positifs ou deux négatifs) produit un résultat avec un signe opposé (MSB invagliné). L'addition de deux nombres de signes différents ne peut jamais produire d'Overflow.

Q : Quel est le résultat de l'application de la procédure du complément à deux sur la valeur zéro (00000000_2) sur 8 bits ?

R : Inversion des bits : 11111111_2. Ajout de 1 : 11111111 + 1 = (1)00000000_2. La retenue sortante 1 est ignorée, le résultat reste 00000000_2, ce qui prouve l'unicité de la représentation du zéro en complément à deux.